

Widget de Chat con RAG

Asistente conversacional sobre documentación pública, embebible en portales de terceros

Reporte técnico y de estado del proyecto

Preparado por: D. Muñoz · Fecha: 30 de julio de 2026 · Estado: Funcional (validado end-to-end)

1. Resumen ejecutivo

Se desarrolló desde cero un **widget de chat flotante con RAG** (*Retrieval-Augmented Generation*) que responde, en lenguaje natural, preguntas de clientes basándose **exclusivamente** en la documentación pública de la empresa (alojada en GitBook). El widget se instala en cualquier portal externo con **una sola línea de código** y está aislado para no interferir con el sitio anfitrión.

El sistema está **construido y validado de punta a punta**: se ingirió la documentación del producto **DPS POS (DigitalPharma)**, se comprobó la búsqueda semántica, la generación de respuestas fundamentadas, la cita de fuentes y la regla anti-invencción. Queda pendiente únicamente el **despliegue a un servidor público** y completar la ingesta del último tramo de páginas (limitación temporal de cuota gratuita, no técnica).

136

PÁGINAS INGERIDAS

498

FRAGMENTOS INDEXADOS

17

MÓDULOS DE CÓDIGO

1

LÍNEA PARA INSTALAR

Valor para el negocio: reduce carga de soporte respondiendo dudas frecuentes 24/7 con información oficial y verificable (con enlaces a la fuente), sin riesgo de "inventar" respuestas y sin trabajo de integración para los portales que lo adopten.

2. Qué hace el producto

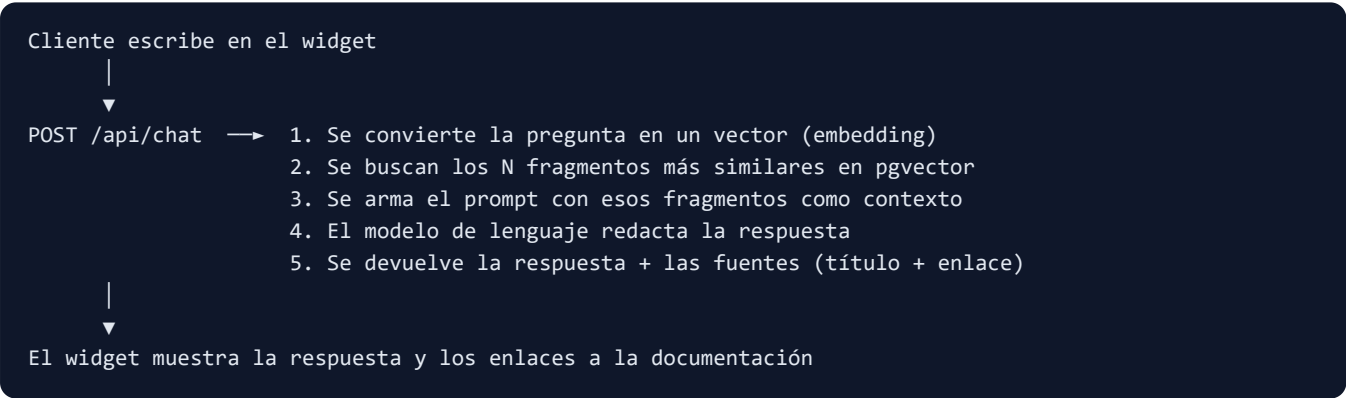
- **Responde en lenguaje natural** preguntas de clientes sobre la documentación.
- **Solo usa información oficial:** recupera los fragmentos más relevantes de la documentación y responde con base en ellos.
- **Cita las fuentes:** cada respuesta incluye enlaces a las páginas de la documentación utilizadas.
- **No inventa:** si la respuesta no está en la documentación, lo dice explícitamente y sugiere contactar a soporte.
- **Se instala en cualquier portal** con una línea, sin que el portal ceda control ni comparta datos sensibles.
- **Escalable a más manuales:** agregar nueva documentación es cambiar una lista de configuración y re-ejecutar la ingesta; no requiere programar.

3. Arquitectura de la solución

El sistema tiene tres piezas, más una base de datos vectorial en la nube:

Componente	Rol	Tecnología
Pipeline de ingesta	Descarga la documentación, la divide en fragmentos, la convierte en vectores y la almacena. Re-ejecutable e idempotente.	Node.js
Backend / API	Recibe la pregunta, busca los fragmentos relevantes, arma el contexto y genera la respuesta con fuentes. Protegido con CORS y límite de peticiones.	Node.js + Express
Widget (frontend)	Botón flotante + panel de chat que se inyecta en el portal, aislado visualmente. Consume la API.	JavaScript autocontenido (Shadow DOM)
Base de datos vectorial	Almacena los fragmentos y sus vectores; permite la búsqueda por similitud semántica.	PostgreSQL + pgvector (Supabase)

Flujo de una pregunta (RAG)



Decisión de arquitectura destacable: el proveedor de Inteligencia Artificial está **desacoplado** tras una interfaz común. Hoy usa **Google Gemini** (capa gratuita), pero se puede cambiar a **Claude** u **OpenAI** para producción **modificando solo variables de configuración, sin tocar el código.**

4. Stack tecnológico

Infraestructura

- **Node.js** (runtime) + **Express** (API)
- **PostgreSQL + pgvector** en **Supabase** (nube, plan gratuito)
- Gestor de paquetes: **pnpm**
- Sin Docker: despliegue directo

Inteligencia Artificial (actual)

- **Embeddings:** Gemini `gemini-embedding-001` (768 dimensiones)
- **Generación:** Gemini `gemini-flash-latest`
- Alternativas listas: OpenAI, Claude

Seguridad y robustez ya implementadas

Medida	Descripción
CORS con lista blanca	Solo los dominios autorizados pueden consumir la API. Nunca abierto a todos.
Límite de peticiones por IP	Protege el endpoint público de abuso (rate limiting configurable).
Regla anti-invencción	Instrucción estricta en el prompt: responder solo con el contexto o admitir que no se sabe.
Aislamiento visual (Shadow DOM)	El widget no choca con los estilos del portal anfitrión ni viceversa.
Ingesta idempotente	Re-ejecutar no duplica datos: actualiza lo cambiado y elimina lo obsoleto.

5. Pruebas realizadas

Prueba	Resultado
Instalación de dependencias y arranque del sistema	✓ Correcto
Conexión a la base de datos y creación automática de tablas/índices	✓ Correcto
Ingesta de la documentación (descarga → fragmentación → vectores → guardado)	✓ 136 páginas / 498 fragmentos
Búsqueda semántica en la base vectorial	✓ Recupera contexto relevante
Generación de respuesta fundamentada + fuentes con enlaces	✓ Correcto
Regla anti-invencción (pregunta fuera de la documentación)	✓ Se niega correctamente
El backend sirve el widget y la página de prueba	✓ Correcto
Prueba de pregunta "en vivo" desde el navegador	✗ Pendiente (ver limitación de cuota)

Ejemplo real validado: ante la pregunta "¿de qué trata la sección Introducción?", el asistente respondió correctamente que el sistema es software para farmacias (gestión de inventarios, trazabilidad de medicamentos, digitalización de procesos) *citando la página exacta*. Ante una pregunta no relacionada ("¿capital de Francia?"), respondió que esa información no está en la documentación.

6. Estado actual y limitaciones

El proyecto está **funcional y validado**. Las limitaciones actuales son **operativas, no de diseño**:

Punto	Situación	Solución
Ingesta incompleta (faltan ~30 páginas)	La capa gratuita de Gemini limita a ~100 vectorizaciones por día y se alcanzó el tope.	Re-ejecutar la ingesta al día siguiente (retoma solo lo que falta) o activar facturación.
Preguntas "en vivo" limitadas hoy	Cada pregunta consume una vectorización; con la cuota agotada, fallan temporalmente.	Se reanuda al reiniciarse la cuota diaria; en producción se resuelve con plan de pago.
Aún en entorno local	Todo corre en la máquina de desarrollo (localhost).	Desplegar el backend a un host público con HTTPS (ver sección 7).

Recomendación para producción: activar facturación en Google Gemini (los límites suben drásticamente y el costo por consulta es muy bajo), o cambiar el proveedor de embeddings a OpenAI. La arquitectura ya lo soporta sin cambios de código.

7. Plan de despliegue e instalación en portales

El widget **no se distribuye por separado**: el mismo backend que responde las preguntas también entrega el archivo del widget. Por lo tanto, publicar el sistema consiste en **desplegar el backend a una URL pública con HTTPS**. La base de datos ya está en la nube y no se mueve.

Pasos

1. Desplegar el backend Node en un host (ej. Render, Railway, Fly.io o un servidor propio), con HTTPS.
2. Configurar las variables de entorno en el panel del host (credenciales de base de datos, clave de IA, modelos y la lista de dominios autorizados).
3. Registrar en la lista blanca de CORS los dominios reales de los portales que instalarán el widget.
4. Entregar a cada desarrollador de portal **una sola línea** para pegar en su sitio.

La única línea que recibe el desarrollador del portal

```
<script src="https://chat.tuempresa.com/widget.js" async></script>
```

El widget detecta automáticamente a qué API llamar según el dominio desde el que se cargó, por lo que el desarrollador externo no configura nada más.

8. Próximos pasos

#	Acción	Responsable / nota
1	Completar la ingesta del tramo restante de páginas	Automático al reanudarse la cuota
2	Decidir plan de IA para producción (facturación Gemini u OpenAI)	Decisión de negocio
3	Desplegar el backend a host público con HTTPS	Desarrollo
4	Definir dominios de portales piloto y registrarlos en CORS	Desarrollo + negocio
5	Instalar la línea del widget en un portal piloto y validar en vivo	Dev del portal
6	(Opcional) Agregar más manuales a la base de conocimiento	Solo configuración

Conclusión: el producto está técnicamente terminado y probado. El camino a producción es corto y consiste principalmente en decisiones de infraestructura (hosting) y de plan de IA, no en desarrollo adicional.